

# Test Results

All tests were run on 2026-05-13 with Flask running on localhost:5000 connected to EC2 MySQL at 18.223.123.14.

---

## API Tests

### test\_rooms.py — Room Search Endpoint

#### Code:

```
import requests

BASE_URL = "http://localhost:5000"

def test_search_returns_200():
    response = requests.get(f"{BASE_URL}/api/rooms/search?day=Monday&starttime=10:00&endtime=12:00&building=PH")
    assert response.status_code == 200, "Search should return 200"
    print("PASS - Search returns 200")

def test_search_returns_list():
    response = requests.get(f"{BASE_URL}/api/rooms/search?day=Monday&starttime=10:00&endtime=12:00&building=PH")
    data = response.json()
    assert isinstance(data, list), "Search should return a list"
    print(f"PASS - Search returns a list with {len(data)} rooms")

def test_search_all_buildings():
    response = requests.get(f"{BASE_URL}/api/rooms/search?day=Monday&starttime=10:00&endtime=12:00")
    assert response.status_code == 200, "Search with no building filter should return 200"
    print("PASS - Search with no building filter works")

def test_search_no_duplicate_rooms():
    response = requests.get(f"{BASE_URL}/api/rooms/search?day=Monday&starttime=10:00&endtime=12:00")
    data = response.json()
    room_codes = [r["room_code"] for r in data]
    assert len(room_codes) == len(set(room_codes)), "No duplicate rooms in search results"
    print("PASS - No duplicate rooms in search results")

def test_search_rooms_have_correct_fields():
    response = requests.get(f"{BASE_URL}/api/rooms/search?day=Monday&starttime=10:00&endtime=12:00&building=PH")
    data = response.json()
    if len(data) > 0:
        room = data[0]
        assert "id" in room
        assert "room_code" in room
        assert "building" in room
        print("PASS - Rooms have correct fields")
```

#### Results:

| Test                                                | Result |
|-----------------------------------------------------|--------|
| Search returns 200                                  | PASS   |
| Search returns a list with 14 rooms                 | PASS   |
| Search with no building filter works                | PASS   |
| No duplicate rooms in search results                | PASS   |
| Rooms have correct fields (id, room_code, building) | PASS   |

All 5 room search tests passed.

---

## test\_schedule.py — Room Schedule Endpoint

### Code:

```
import requests

BASE_URL = "http://localhost:5000"

def test_schedule_returns_200():
    response = requests.get(f"{BASE_URL}/api/rooms/1/schedule?day=Monday")
    assert response.status_code == 200, "Schedule should return 200"
    print("PASS - Schedule returns 200")

def test_schedule_returns_list():
    response = requests.get(f"{BASE_URL}/api/rooms/1/schedule?day=Monday")
    data = response.json()
    assert isinstance(data, list), "Schedule should return a list"
    print(f"PASS - Schedule returns a list with {len(data)} entries")

def test_schedule_has_correct_fields():
    response = requests.get(f"{BASE_URL}/api/rooms/1/schedule?day=Monday")
    data = response.json()
    if len(data) > 0:
        entry = data[0]
        assert "course_name" in entry
        assert "instructor" in entry
        assert "start_time" in entry
        assert "end_time" in entry
        print("PASS - Schedule entries have correct fields")

def test_schedule_times_are_strings():
    response = requests.get(f"{BASE_URL}/api/rooms/1/schedule?day=Monday")
    data = response.json()
    if len(data) > 0:
        entry = data[0]
        assert isinstance(entry["start_time"], str)
        assert isinstance(entry["end_time"], str)
        print("PASS - Times are returned as strings")

def test_invalid_room_returns_empty():
    response = requests.get(f"{BASE_URL}/api/rooms/999999/schedule?day=Monday")
    assert response.status_code == 200
    data = response.json()
    assert data == []
    print("PASS - Invalid room ID returns empty list")
```

### Results:

| Test                                                                                 | Result |
|--------------------------------------------------------------------------------------|--------|
| Schedule returns 200                                                                 | PASS   |
| Schedule returns a list with 4 entries                                               | PASS   |
| Schedule entries have correct fields (course_name, instructor, start_time, end_time) | PASS   |
| Times are returned as strings not timedelta objects                                  | PASS   |
| Invalid room ID returns empty list not a crash                                       | PASS   |

All 5 schedule tests passed.

---

## test\_admin.py — Admin Auth Endpoint

### Code:

```
import requests

BASE_URL = "http://localhost:5000"

def test_upload_no_token_returns_401():
    response = requests.post(f"{BASE_URL}/api/admin/semester")
    assert response.status_code == 401
    data = response.json()
    assert data["message"] == "Unauthorized"
    print("PASS - Upload with no token returns 401")

def test_upload_invalid_token_returns_401():
    headers = {"Authorization": "Bearer invalidtoken123"}
    response = requests.post(f"{BASE_URL}/api/admin/semester", headers=headers)
    assert response.status_code == 401
    data = response.json()
    assert data["message"] == "Invalid token"
    print("PASS - Upload with invalid token returns 401")
```

### Results:

| Test                                                | Result |
|-----------------------------------------------------|--------|
| Upload with no token returns 401 Unauthorized       | PASS   |
| Upload with invalid token returns 401 Invalid token | PASS   |

All 2 admin auth tests passed.

---

## Database Tests (run in MySQL on EC2 via PuTTY)

### Code:

```
-- Test 1: Check data was loaded
SELECT 'rooms count' AS test, COUNT(*) AS result FROM rooms;
SELECT 'schedules count' AS test, COUNT(*) AS result FROM schedules;
SELECT 'semesters count' AS test, COUNT(*) AS result FROM semesters;

-- Test 2: Check for duplicate rooms (should return 0 rows)
SELECT room_code, building, COUNT(*) AS count
FROM rooms
GROUP BY room_code, building
HAVING COUNT(*) > 1;

-- Test 3: Check schedules are linked to valid rooms (should return 0)
SELECT COUNT(*) AS orphaned_schedules
FROM schedules
WHERE room_id NOT IN (SELECT id FROM rooms);

-- Test 4: Check times are stored correctly
SELECT room_id, start_time, end_time
FROM schedules
LIMIT 5;

-- Test 5: Check a specific room search works
SELECT DISTINCT rooms.id, rooms.room_code, rooms.building
FROM rooms
WHERE rooms.id NOT IN (
    SELECT room_id FROM schedules
    WHERE day = 'M'
```

```

        AND start_time < '12:00:00'
        AND end_time > '10:00:00'
    )
    AND rooms.building = 'PH'
LIMIT 5;

```

## Results:

| Test                   | Expected        | Result                                 | Status |
|------------------------|-----------------|----------------------------------------|--------|
| Rooms count            | > 0             | 256                                    | PASS   |
| Schedules count        | > 0             | 2041                                   | PASS   |
| Semesters count        | 1               | 1                                      | PASS   |
| Duplicate rooms        | 0 rows          | Empty set                              | PASS   |
| Orphaned schedules     | 0               | 0                                      | PASS   |
| Times stored correctly | HH:MM:SS format | 09:10:00, 12:00:00 etc.                | PASS   |
| Room search query      | 5 rows          | PH 110, PH 202, PH 302, PH 112, PH 012 | PASS   |

**All 7 database tests passed.**

---

## Summary

| Test File         | Tests Run | Passed | Failed |
|-------------------|-----------|--------|--------|
| test_rooms.py     | 5         | 5      | 0      |
| test_schedule.py  | 5         | 5      | 0      |
| test_admin.py     | 2         | 2      | 0      |
| test_database.sql | 7         | 7      | 0      |
| Total             | 19        | 19     | 0      |

**All 19 tests passed successfully.**